```
/* geospatial analysis */
/* Load fqhc and survey data */
proc import datafile="/home/u59438703/CINJ/fqhc_sites.csv" out=fqhc_sites dbms=csv replace;
    getnames=yes;
run;

proc import datafile="/home/u59438703/CINJ/survey_data.xlsx" out=survey_data dbms=xlsx replace;
    getnames=yes;
run;

proc contents data=fqhc_sites;
run;

/* Step 0: Import FQHC and Survey Data */
proc import datafile="/home/u59438703/CINJ/fqhc_sites.csv"
    out=fqhc_sites dbms=csv replace;
    getnames=yes;
run;

proc import datafile="/home/u59438703/CINJ/survey_data.xlsx"
    out=survey_data dbms=xlsx replace;
    getnames=yes;
run;

/* Step 1: Clean and Prepare Population and Coordinate Data */
data tracts_clean;
    set fqhc_sites;
    if missing(lat) or missing(lon) or missing(population) then delete;
    tract_lat = lat;
    tract_lon = lon;
    population_num = input(compress(population, ','), best32.);
    pcp_count_num = input(compress(pcp_count, ','), best32.);
run;

/* Step 2: Create Distance Matrix */
proc sql;
    create table distance_matrix as
    select
        a.site_id,
        a.tract_id,
        a.pcp_count_num,
        b.tract_id as demand_tract_id,
        b.tract_lat as demand_lat,
        b.tract_lon as demand_lon,
        b.population_num
    from tracts_clean as a, tracts_clean as b;
quit;

/* Step 3: Compute Haversine Distance and Tiered Decay Weight */
data distance_matrix;
    set distance_matrix;

    /* Convert degrees to radians */
    lat1_rad = tract_lat * constant('pi') / 180;
    lon1_rad = tract_lon * constant('pi') / 180;
    lat2_rad = demand_lat * constant('pi') / 180;
    lon2_rad = demand_lon * constant('pi') / 180;

    /* Haversine formula */
    dlat = lat2_rad - lat1_rad;
    dlon = lon2_rad - lon1_rad;

```

```
    a = sin(dlat/2)**2 + cos(lat1_rad) * cos(lat2_rad) * sin(dlon/2)**2;
    c = 2 * arsin(sqrt(a));
    distance_km = 6371 * c;

    if distance_km < 0.1 then distance_km = 0.1;

    /* Power-decay weights (tiered) */
    if distance_km <= 10 then decay_weight = 1;
    else if distance_km <= 20 then decay_weight = (10 / distance_km)**1.5;
    else if distance_km <= 30 then decay_weight = (10 / distance_km)**2;
    else decay_weight = 0;

    weighted_pop = population_num * decay_weight;
run;

/* Step 4: Compute Supply-to-Population Ratio (Rj) */
proc sql;
    create table site_supply as
    select site_id,
           pcp_count_num,
           sum(weighted_pop) as weighted_demand
    from distance_matrix
    group by site_id;
quit;

data site_supply;
    set site_supply;
    if weighted_demand > 0 then Rj = pcp_count_num / weighted_demand;
    else Rj = .;
run;

/* Step 5: Merge Rj into Distance Matrix and Compute Accessibility */
proc sql;
    create table distance_with_Rj as
    select a.*, b.Rj
    from distance_matrix as a
    left join site_supply as b
    on a.site_id = b.site_id;
quit;

data valid_distances;
    set distance_with_Rj;
    if not missing(Rj) and not missing(decay_weight);
    partial_access = Rj * decay_weight;
run;

proc sql;
    create table accessibility_scores as
    select demand_tract_id as tract_id,
           sum(partial_access) as accessibility_score
    from valid_distances
    group by demand_tract_id;
quit;

/* Step 6: Export Scores and Show Sample */
proc print data=accessibility_scores(obs=10); run;

proc export data=accessibility_scores
    outfile="/home/u59438703/CINJ/accessibility_scores_tiered.csv"
    dbms=csv replace;
run;

/* Step 7: Visualization */
```

```
proc sgplot data=accessibility_scores;
    histogram accessibility_score / binwidth=0.2 transparency=0.2;
    density accessibility_score / type=normal;
```